

HW 4

P1

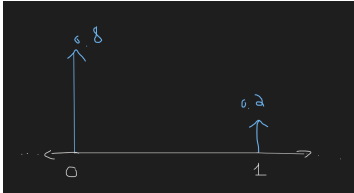
1. Discrete probability distributions [35 pts]

- a. Draw the pmf and cdf of a Bernoulli random variable with $p = 0.2$. [2 pts]

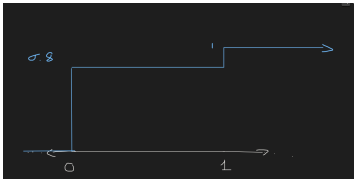
Bernoulli distribution:

$$P(X = 1) = p$$
$$P(X = 0) = 1 - p$$

PDF:



CDF:



This is a delta at each of the two binary choices. Those are the only possible two, so any other value on the graph is zero (do we need to show that it goes to inf, or just leave those two points only since that is the whole sample space?). The CDF just adds up the deltas that are to the left of whatever point.

- b. In your jupyter notebook, create a function that simulates N draws from a Bernoulli random variable with parameter p and returns the result as a numpy array. You can do this by drawing a random variable uniformly distributed between 0 and 1 (see [numpy.random.uniform](#)), and then checking to see if it is less than p . [3 pts]

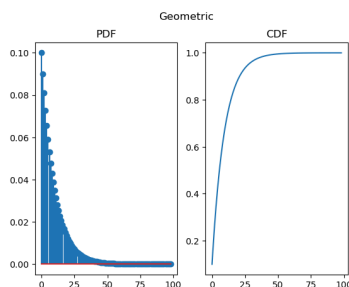
This shows the percentage that are ≤ 0.2 , which for large numbers is 0.2%.

- c. Derive an expression for the cdf of a Geometric random variable with parameter p , and then plot the pmf and cdf for $p = 0.1$. [5 pts]

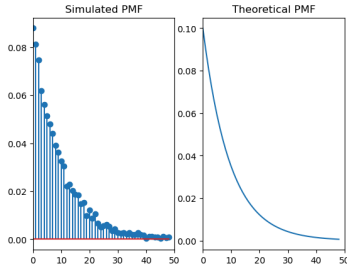
$$p_x(X = k) = (1 - p)^{k-1}p$$
$$k = 1, 2, 3, 4, 5$$

This gives the CDF as

$$P_x = \sum_{k=0}^x p(1 - p)^{k-1}$$



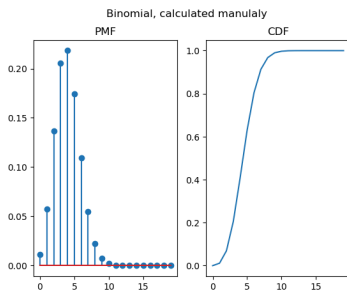
- d. In your jupyter notebook, create a function that simulates N draws from a Geometric random variable with parameter p . A single draw from a geometric distribution can be done by simulating a series of Bernoulli random variables until a “success” occurs and recording how many trials it took. Once your function is implemented, simulate a reasonably large number of draws from a geometric distribution with $p = 0.1$ and create a stem plot that shows the empirical pmf based on simulation. Show the theoretical distribution (from part c) side-by-side to compare. [10 pts]



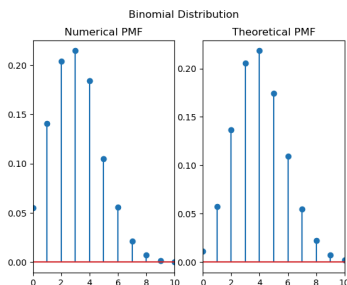
- e. In your jupyter notebook, plot the pmf and cdf of a Binomial distribution with $n = 20$ and $p = 0.2$. You may take a purely computational approach to this part (rather than a theoretical derivation). Note that $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ indicates the number of ways to select k out of n elements. [5 pts]

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

for $k = 0, 1, 2, \dots, n$

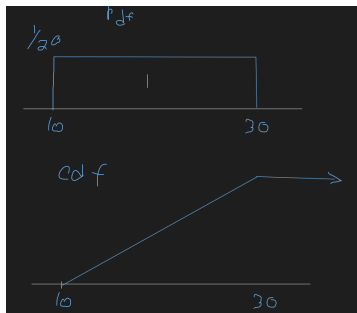


- f. In your jupyter notebook, create a function that simulates N draws from a Binomial distribution with parameters n and p . A single draw from a Binomial distribution can be done by simulating n Bernoulli random variables and counting the number of ones. Once your function is implemented, simulate a reasonably large number of draws from a binomial distribution with $n = 20$ and $p = 0.2$, and create a stem plot that shows the empirical pmf based on simulation. Show the theoretical pmf (from part e) side-by-side to compare. [10 pts]



2. Continuous probability distributions. [30 pts]

- a. Sketch the pdf and cdf of a random variable X distributed uniformly between 10 and 30. Calculate an expression for $P(\gamma_{\text{lower}} < X < \gamma_{\text{upper}})$. Your expression should consider any values of γ_{lower} and γ_{upper} where $\gamma_{\text{lower}} < \gamma_{\text{upper}}$. [4 pts]



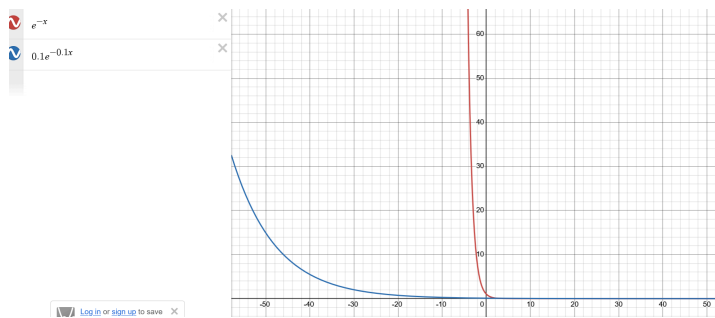
$$P(\gamma_{\text{lower}} \leq x \leq \gamma_{\text{upper}}) = \frac{\gamma_{\text{upper}} - \gamma_{\text{lower}}}{20}$$

- b. Derive an expression for the cdf of an exponential random variable with parameter λ . Sketch the pdf and cdf for the cases $\lambda = 1$ and $\lambda = 0.1$. Calculate an expression for $P(a < X < b)$ where $0 < a < b$. [6 pts]

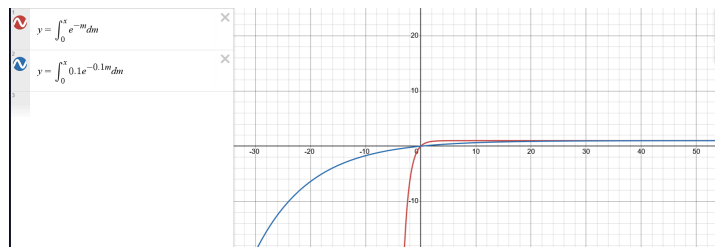
$$f_X(x; \lambda) = \begin{cases} \lambda e^{-\lambda x}, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

$$F_X(x; \lambda) = \int_0^x \lambda e^{-\lambda t} dt$$

PDFs:

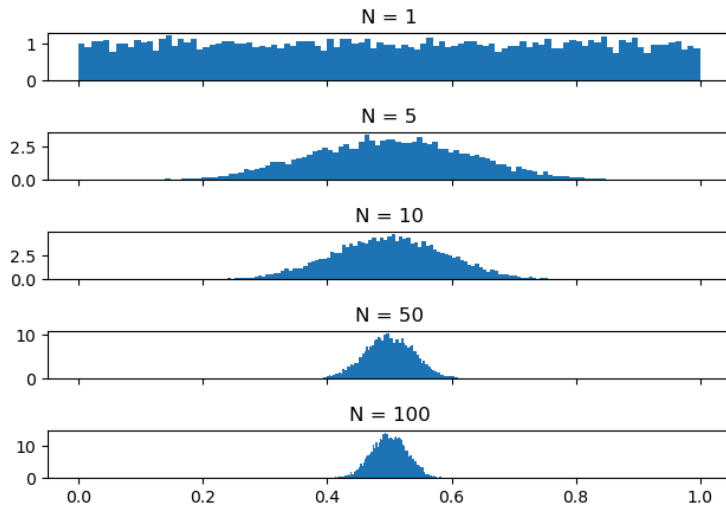


CDFs:



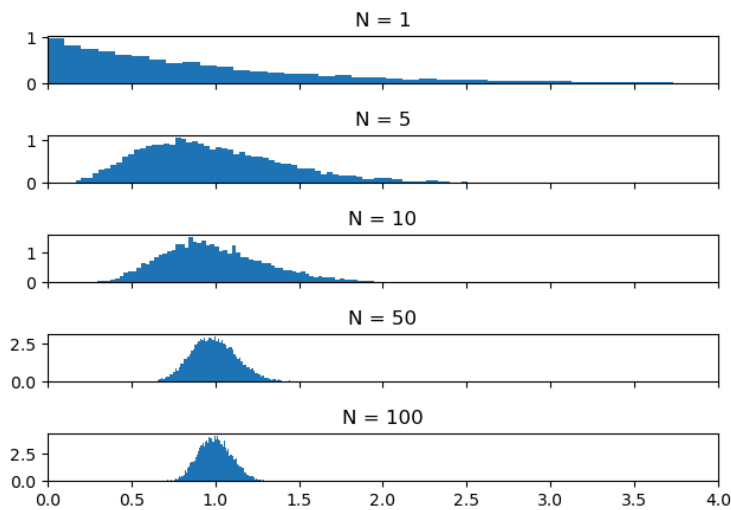
- c. We will experimentally verify the Central Limit Theorem in this part. Let X be the average of N random draws from a uniform distribution between 0 and 1. Draw lots of samples for $N = 1$ and show a normalized histogram (i.e. relative frequency) of the samples using a bin spacing of 0.01 between 0 and 1. Repeat for several larger values of N and show the corresponding normalized histograms. Visually show how the distribution converges to a Gaussian distribution as N gets larger. [10 pts]

Central Limit Theorem



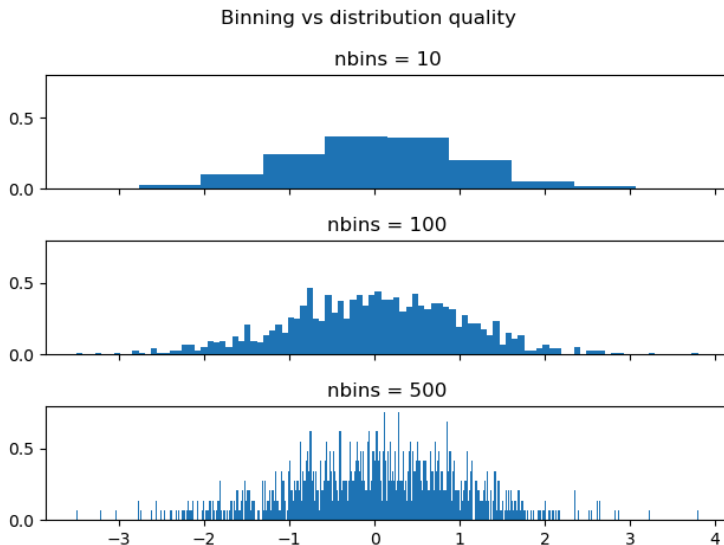
- d. Let Y be the average of N random draws from an exponential distribution with parameter $\lambda = 1$. You may use `numpy.random.exponential` to simulate draws from an exponential distribution. Draw lots of samples for $N = 1$ and show a normalized histogram (i.e. relative frequency) of the samples using an appropriate bin spacing and range. Repeat for several larger values of N and show the corresponding normalized histograms. Visually show how the distribution converges to a Gaussian distribution as N gets larger. Note that it doesn't matter what the initial distribution is – the sum (or average) of lots of independent random variables will always end up looking like a Gaussian! [3 pts]

Central Limit Theorem on Exponential Decays



- e. Draw lots of samples from a normal distribution using the `numpy.random.normal` function. Then generate three histograms using three different bin spacings. Explain the relationship between the histogram bin size, the normalized histogram value (i.e. relative frequency), and the pdf value, and validate this with your empirical histograms. Include relevant plots to support your answer. [7 pts]

For this I used $n_{\text{samples}} = 1000$, so that I could see what happened as the number of bins approached a significant portion of the number of samples



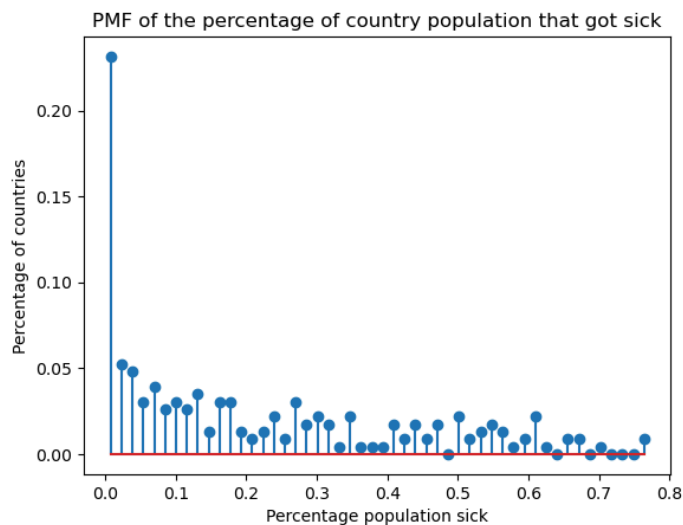
We can see the gaussian shape well for a medium number of bins, but loose coherence when we have too many bins and don't tell the story well with too few relative to the total number of data points. Because these are continuous distribution, we care about the ranges of values - so there is a notion of density. If we have too many bins, then data points can all fit neatly into their own category - so they don't stack neatly as they do when they are clumped. The PDF for each of these follow the shape of a pure continuous gaussian, $\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}$ (yes that was memorized) - but these are really discrete points that were sampled and stored in a bit representation. Counterintuitively, with more bins we see many more values with nothing but ALSO many values with abnormally high peaks - i.e. we don't see any bins above 0.5 for the first two, but we see a few for the third.

3 Discrete-valued scalar data 20 pts

- a. Find, scrape, or collect a set of discrete-valued scalar data for some topic of interest to you that involves randomness (e.g. sports scores, climate, etc). Your data should simply be a large set of discrete scalar values. Kaggle is a useful website for finding interesting datasets. Briefly describe your data in the jupyter notebook and show some basic summary information. You may use pandas to read in and manipulate the data as needed. Please do not use the same dataset as your classmates. [8 pts]

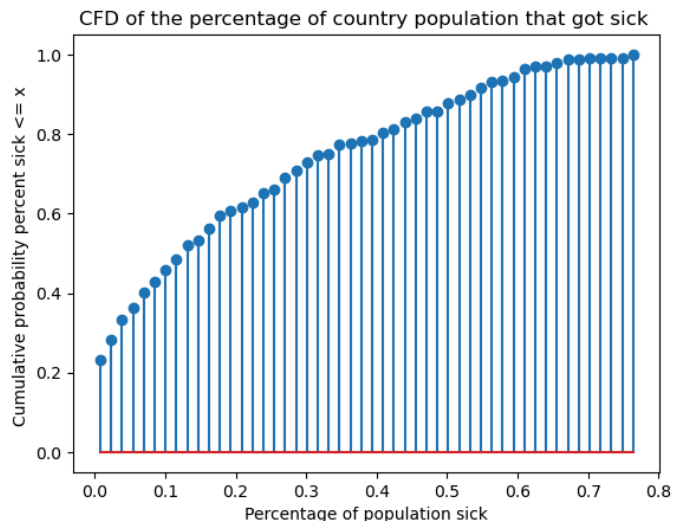
I'm using a dataset on covid cases globally. The random variable is the percentage of population sick (i.e. pick a random country). There are many countries that fit into different bins for percentage of population sick.

- b. Implement a function from scratch in numpy that takes in the raw data and calculates an empirical pmf or cdf (your choice). Between pmf and cdf, which is easier or more straightforward to compute from the raw data? Use your function to display the empirical pmf or cdf as a stem plot. [6 pts]



The pmf seems far easier, because we can just make bins. Going from pmf to cdf is just a cumulative sum, so one line. Going backwards would be a pairwise difference, but getting all the data in a range feels more convoluted. It would just be bigger and bigger bins, so not that bad actually, just a little bit weirder.

- c. Implement a function that calculates the other characterization of the data distribution. So, if you chose to implement the pmf in part b then implement a function that calculates the empirical cdf (and vice versa). You may use your function in part b in your implementation. [2 pts]



- d. Comment on the distribution that you observe. What do you see in the data?

Explain how the distribution reveals insights into the data. [2 pts]

Most countries had very small percentages of people who were sick - 60 percent of the population had under 20 percent sickness. There are some where basically everyone got sick, but i think that depends on population size - if its a tiny country (or had horrible regulation) the number is high.

- e. Define an event of interest on the dataset and calculate the probability of that event on the empirical distribution. [2 pts]

I think that good events for this dataset are the chance that you had less than x% sick, which is just the CDF, or 0 % sick which is just the pmf at zero.

There is a 48% chance of getting under 11% of the population sick. There is a 23% chance of having negligibly few cases.

A better even for this question: What is the probability of falling between 0.2 and 0.4 % of the population? That seems really bad for the country.

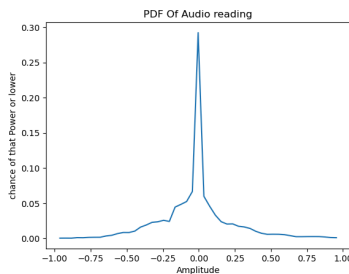
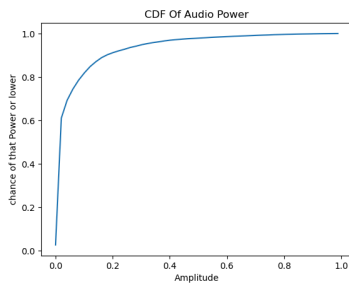
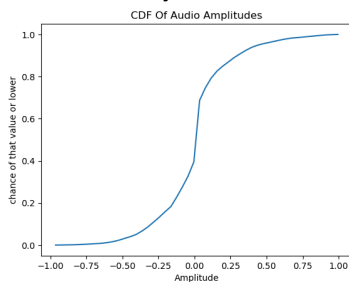
There is a 19% chance of falling between 20 and 40 percent population infection.

4

4. Continuous-valued scalar data [15 pts]

- Find, scrape, or collect a set of continuous-valued scalar data that involves randomness. One easy way to do this is to simply take the measurements from some device (e.g. voltage measurements, audio recording, etc). Your data should simply be a large set of continuous-valued scalar values (devices with high resolution can be considered continuous-valued for our purposes). Briefly describe your data in the jupyter notebook. [5 pts]
- Implement a function from scratch in numpy that takes in the raw data and calculates an empirical pdf or cdf (your choice). Between pdf and cdf, which is easier or more straightforward to compute from the raw data? Use your function to display the empirical pdf or cdf as a plot. [6 pts]

It is easy to compute either, I copied my manual PMF code and modified it to be a manual CDF for fun. The other cdf function i had done was just a cumsum of pdf.



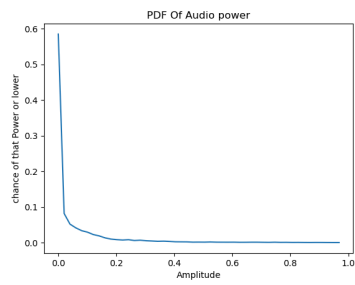
- Comment on the distribution that you observe. What do you see in the data? Explain how the distribution reveals insights into the data. [2 pts]

Most of the audio has low power, so the cdf goes up really fast if we are looking at that. If we look at values, we see almost no extreme -1 or +1, and a lot centered close to zero.

This also looks like there are more quiet moments in the recording, which would have near zero amplitude - normally this wouldn't have such a strong peak around zero (even though oscillations have to cross zero, it has values equally across the sine wave)

- Define an event of interest on the dataset and calculate the probability of that event on the empirical distribution. [2 pts]

An event could be that the signal power is within the 1/2 power width. Because the signal is normalized from 0 to 1, (and getting power is squared), this is data above ~ 0.70 .



0.73 % chance of getting a power $\geq \sqrt{2}$. This is funky data, but that looks real.